Lecture Notes: Model Selection

CIC1205 - Machine Learning

Eduardo Bezerra

March 13, 2026

0 Contents

1 Introduction **3**
1.1 Motivation . . . 3
1.2 Hyperparameters . . . 3
1.3 Definition . . . 4
1.4 Goals of Model Selection . . . 4
1.5 Steps in the Model Selection Process . . . 4
1.6 Relation to Model Evaluation . . . 4
1.7 Challenges . . . 5
1.8 Model Selection Methods . . . 5

2 Testing on Training Data **5**

3 Two-Way Holdout **6**
3.1 Procedure . . . 6
3.2 Two-Way Holdout and Hyperparameter Tuning . . . 7

4 Three-Way Holdout **8**
4.1 Motivation . . . 8
4.2 Procedure . . . 8
4.3 Worked Example: Optimistic Bias from Test-Set Reuse . . . 9
4.4 Typical Split Proportions . . . 11
4.5 Advantages and Limitations . . . 13

5 k-Fold Cross-Validation **14**
5.1 The Big Picture: Where Cross-Validation Fits . . . 14
5.2 Motivation . . . 15
5.3 Concept and Procedure . . . 16
5.4 Advantages and Limitations . . . 16
5.4.1 Advantages . . . 16
5.4.2 Limitations . . . 17
5.5 Choice of k . . . 17
5.6 Relation to the Test Set . . . 18
5.7 Cross-Validation for Hyperparameter Tuning . . . 18
5.8 Cross-Validation for Algorithm Selection . . . 19
5.9 When Cross-Validation Is Not Enough . . . 19

6 Nested Cross-Validation **19**
6.1 Motivation . . . 19
6.2 Outer Loop: Evaluation . . . 20
6.3 Inner Loop: Hyperparameter Tuning . . . 20
6.4 Illustrative Example . . . 21
6.5 Worked Example: Tracing Nested CV Step by Step . . . 22
6.6 What About the Final Model? . . . 24

7 Final Remarks **25**
7.1 Beyond the Methods Presented . . . 25
7.2 Which Method Should You Use? . . . 27

7.3 Do You Always Need a Separate Test Set? 27
7.4 Choosing the Right Metric 28
7.5 Keep the Test Set in a Safe 28
7.6 Final Takeaways 29

1 Introduction

1.1 Motivation

When fitting a machine learning model, there are always two main questions to answer experimentally:

1. Which **learning algorithm** is best suited for the task?
2. Which **combination of hyperparameters** produces the best generalization performance?

Choosing the algorithm and tuning its hyperparameters together is called **model selection**. This step is essential in any ML pipeline because it determines how well the final model will perform on unseen data.

1.2 Hyperparameters

Two fundamentally different kinds of values determine a model's behavior. The first kind, **model parameters**, are learned automatically from data during training: examples include the weights of a neural network or the coefficients of a linear regression. The second kind, **hyperparameters**, are values set by the practitioner or training procedure rather than learned as ordinary model parameters from the training objective; they govern how the algorithm searches for good parameters and how much complexity the resulting model is allowed to have.

Hyperparameter

A **hyperparameter** is a configuration value chosen by the practitioner (or by an external search procedure) that influences how the learning algorithm fits model parameters to the data.

This distinction matters because selecting a model is, in large part, selecting its hyperparameters. Two instances of the same algorithm trained on the same data but with different hyperparameter values can exhibit very different generalization behavior.

Examples.

- In linear models: regularization constant (λ).
- In neural networks: learning rate, number of layers, dropout rate.
- In tree-based models: maximum depth, minimum samples per leaf.
- In k-Nearest Neighbors: the number of neighbors k.

These notes focus on *how to evaluate and compare* candidate models and hyperparameter configurations reliably. The complementary question (how to systematically *search* the hyperparameter space using strategies such as grid search, random search, and Bayesian optimization) is treated in the companion notes on Hyperparameter Search Strategies.

1.3 Definition

Model Selection

Model selection is the process of comparing multiple candidate models and identifying the one that achieves the best estimated generalization performance according to a chosen evaluation metric.

The word "model" here can refer either to different algorithms (e.g., logistic regression, support vector machines, decision trees) or to different configurations of the same algorithm with distinct hyperparameter values.

1.4 Goals of Model Selection

Model selection aims to achieve two complementary objectives:

- **Algorithm selection:** choose the most appropriate learning algorithm for the problem.
- **Hyperparameter tuning:** choose the most appropriate hyperparameter configuration for a given algorithm.

In practice, both steps are often interleaved: while evaluating several algorithms, we may also search within each algorithm for its best hyperparameters.

1.5 Steps in the Model Selection Process

Model selection is an iterative and empirical process composed of two main steps:

1. **Fit and evaluate candidate models.** For each algorithm or hyperparameter configuration, train the model on a subset of the data and estimate its performance using a separate validation or test subset.
2. **Select the best model.** Choose the candidate that yields the highest generalization performance according to a specific evaluation metric (e.g., accuracy, F1 score, RMSE, or AUC).

Key idea. The quality of model selection depends on how reliably we estimate generalization error. The evaluation procedure (holdout, cross-validation, bootstrap) directly affects this reliability.

1.6 Relation to Model Evaluation

Model selection and model evaluation are related but distinct tasks:

- **Model evaluation** estimates how good a model is.
- **Model selection** decides which model to keep.

Evaluation provides the quantitative evidence used in selection; selection uses that evidence to choose among alternatives.

1.7 Challenges

Model selection is subject to two main sources of error:

- **Optimistic bias:** when the same test set is used to choose the model and to estimate its final performance.
- **Variance:** when the selected model's performance estimate depends heavily on a particular random data split.

To mitigate these issues, we need careful data partitioning and proper validation strategies, explored in the sections that follow.

1.8 Model Selection Methods

There are several practical methods for estimating generalization error and comparing candidate models. These methods differ mainly in how the available data are partitioned into training, validation, and test subsets:

Method	Reliability	Cost
Test on training data	None (incorrect)	Minimal
Two-way holdout	Low–moderate	Low
Three-way holdout	Moderate	Low
k-fold cross-validation	High	Moderate ($\times k$)
Bootstrap-based estimation	High	High

Each method represents a trade-off between reliability and computational cost.

Entry Ticket 1

In your own words, explain why model selection and model evaluation require *different* data subsets. What can go wrong if both activities use the same held-out set?

Data Terminology. In the following sections, we often refer to the union of the training and validation sets as the **development set**. This distinction is crucial: the development set is where we "work" on the model (fitting and tuning), while the **test set** remains a separate entity used only for the final, unbiased evaluation.

2 Testing on Training Data

As a first idea, one might attempt to evaluate the model on the same data used for training. This is a tempting shortcut, but it is fundamentally incorrect.

Why it fails

- The model has already seen those examples, so the error will be artificially small.

- It does not measure how the model performs on unseen data.
- It leads to overoptimistic estimates of generalization.

Analogy. Testing on the training data is like asking your mother if you are beautiful: you will always get a biased answer.

Example. A model trained and evaluated on the same dataset may show 99% accuracy, yet perform poorly on new samples. This happens because the training error $\hat{E}_{\text{train}}$ underestimates the true generalization error E_{gen}.

Rule

Never rely on the training data for evaluation or model selection. The test set must consist of examples the model has *never seen* during training.

3 Two-Way Holdout

The simplest correct evaluation procedure divides the available data into two disjoint subsets with clearly separated roles:

- **Training set** ($\mathcal{D}_{\text{train}}$): used exclusively to fit the model parameters.
- **Test set** ($\mathcal{D}_{\text{test}}$): used exclusively to estimate how well the trained model generalizes to unseen examples.

For i.i.d. data, the two subsets are typically formed by random splitting and must be mutually exclusive and collectively exhaustive:

$$\mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{test}} = \mathcal{D}, \qquad \mathcal{D}_{\text{train}} \cap \mathcal{D}_{\text{test}} = \varnothing.$$

A typical split reserves 70–80% of the data for training and the remainder for testing, though the right proportion depends on dataset size (see Section **??**).

3.1 Procedure

The two-way holdout follows a straightforward sequence of steps, regardless of the learning algorithm or task type:

1. **Split.** Randomly partition $\mathcal{D}$ into $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{test}}$.
2. **Train.** Fit the model on $\mathcal{D}_{\text{train}}$ alone, minimizing a task-appropriate training loss. For regression this is typically mean squared error; for classification it may be cross-entropy or another suitable criterion.
3. **Evaluate.** Apply the trained model to $\mathcal{D}_{\text{test}}$ and compute a performance metric suited to the task: accuracy or F1 for classification, RMSE or MAE for regression, AUC for ranking problems, and so on.
4. **Report.** The test-set metric is the estimate of generalization performance.

The procedure is identical whether the task is classification, regression, or any other supervised learning problem. What changes between tasks is the loss used during training and the metric used during evaluation, not the data-splitting logic itself.

Advantages. Conceptually simple, easy to implement, and computationally inexpensive: the model is trained only once.

Limitations. The performance estimate depends entirely on a single random partition. If the split happens to place easy examples in the test set or hard examples in the training set, the reported metric will not reflect true generalization ability. For small datasets this sensitivity to the random split can be pronounced.

3.2 Two-Way Holdout and Hyperparameter Tuning

The two-way holdout becomes problematic as soon as hyperparameter selection enters the picture. Suppose we are training a classifier and want to choose the best regularization strength C from the set $\{0.001, 0.01, 0.1, 1, 10\}$. A tempting approach is:

1. For each value of C, train a model on $\mathcal{D}_{\text{train}}$.
2. Compute the test-set error for each trained model.
3. Select the value C^* that minimizes the test error and report that error as the model's generalization performance.

Problem. This workflow uses the test set for two distinct purposes simultaneously: *selecting* the best hyperparameter value and *reporting* the final performance. Once the test set has been used to make a selection decision, it is no longer a neutral judge. The winning configuration was chosen precisely because it happened to score best on those particular test examples; the reported error is therefore **optimistically biased**.

Why the bias grows with the number of candidates. Each candidate model evaluated on the test set constitutes one implicit "use" of that set. If five values of C are compared, the test set has effectively been queried five times. The more candidates we compare, the greater the chance that one of them scores well by luck, and the more the reported error underestimates true generalization error. The same logic applies regardless of whether the hyperparameter is a regularization constant, a tree depth, a learning rate, or a kernel bandwidth.

Solution. The root cause is that the test set is asked to play two incompatible roles at once. The remedy is to introduce a third subset, a **validation set**, dedicated exclusively to hyperparameter selection. The test set is then reserved for a single final evaluation only, after all modeling decisions have been made. This leads naturally to the three-way holdout described in the next section.

Entry Ticket 2

A practitioner trains 10 different models (varying regularization strength) and picks the one with the lowest test-set error, then reports that error as the model's per-

formance.

1. Why is this estimate optimistically biased?
2. How many times has the test set effectively been "used" in this process?
3. What should the practitioner have done instead?

4 Three-Way Holdout

4.1 Motivation

The two-way holdout provides only a single estimate based on one random split. If the same test set is also used to choose hyperparameters, the generalization estimate becomes biased. To correct this, the dataset is divided into **three** disjoint parts:

- **Training set:** used to learn the model parameters.
- **Validation set:** used to tune hyperparameters and compare algorithms.
- **Test set:** used only *once*, to obtain a reliable estimate of the final model's generalization error.

4.2 Procedure

Given the dataset $\mathcal{D}$:

1. Randomly partition it into $\mathcal{D}_{\text{train}}$, $\mathcal{D}_{\text{val}}$, and $\mathcal{D}_{\text{test}}$.
2. Train a candidate model on $\mathcal{D}_{\text{train}}$.
3. Evaluate it on $\mathcal{D}_{\text{val}}$ to compute the validation error $E_{\text{val}}(\theta)$.
4. Repeat steps 2–3 for several hyperparameter values or algorithms.
5. Choose the configuration that yields the lowest E_{val}.
6. Retrain the chosen model on the combined training + validation data and evaluate it *once* on $\mathcal{D}_{\text{test}}$.

Goal. The validation set guides model selection; the test set provides a reliable estimate of generalization performance.

Figure 1 illustrates the three-way holdout strategy. The full dataset $\mathcal{D}$ is partitioned into three disjoint subsets: a training set (e.g., 70 %), a validation set (15 %), and a test set (15 %). Each subset has a distinct and non-overlapping role. The training set is used exclusively to fit the model parameters. The validation set is used to compare candidate models and select the best hyperparameter configuration; because it is separate from the training set, the validation error provides a less biased estimate of generalization than the training error would. The test set, highlighted in red, is reserved for a single final evaluation performed only after all modeling decisions (including hyperparameter tuning) have been fully committed to. Using the test set at any earlier stage, or consulting it more than once, would contaminate the final performance estimate and render it optimistically biased.

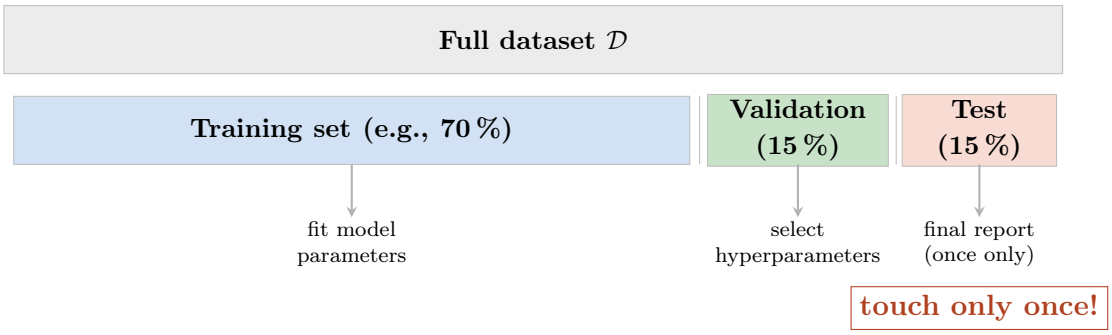


Figure 1: Three-way holdout. The validation set guides hyperparameter selection; the test set is reserved for a single final evaluation.

As an example, consider predicting a target variable y from a single feature x using a **Decision Tree Regressor** with hyperparameter `max_depth`. Shallow trees produce smooth, high-bias approximations; deep trees adapt closely to training data. We divide the data into training, validation, and test sets and train models with `max_depth` varying from 1 to 10. The training error decreases monotonically as the tree deepens. The validation error follows a U-shaped curve: it improves up to the optimal depth, then rises as the model begins to overfit. Figure 2 shows the *validation curve*[1] obtained when tuning the `max_depth` hyperparameter of a decision tree. The blue curve (training error) decreases monotonically as `max_depth` increases: a deeper tree has more capacity and can fit the training data ever more closely, eventually memorizing it almost perfectly. The red curve (validation error) tells a different story; it follows a U-shape, improving initially as the tree gains enough capacity to capture the genuine signal in the data, but then rising again once the tree becomes deep enough to start fitting noise. For `max_depth` values of 1 to 4, both errors are high, indicating that the model underfits: it is too shallow to represent the underlying pattern. Beyond `max_depth` $= 5$, the training error continues to fall while the validation error climbs, the hallmark of overfitting. The green dot marks `max_depth` $= 5$ as the optimal configuration, the point at which the validation error reaches its minimum and the bias–variance trade-off is best balanced. This is the value that would be selected by the three-way holdout or cross-validation procedure, after which the test set is consulted exactly once to report the final performance of the chosen model.

Key takeaway. The validation set guides the *choice* of model complexity; the test set measures the *quality* of that choice. Using the test data during selection would contaminate the evaluation and produce an overly optimistic estimate.

4.3 Worked Example: Optimistic Bias from Test-Set Reuse

Consider the following scenario. A practitioner has a dataset of 1 000 examples and uses a 70/30 train–test split (no validation set). She trains a decision tree with `max_depth` varying from 1 to 5, computes the test MSE for each, picks the model with the lowest test MSE, and reports that number as the model's generalization error.

[1] A **validation curve** is a plot of training error and validation error as a function of a single hyperparameter value, with all other hyperparameters held fixed. It is used to diagnose underfitting and overfitting and to identify the hyperparameter value that minimizes generalization error.

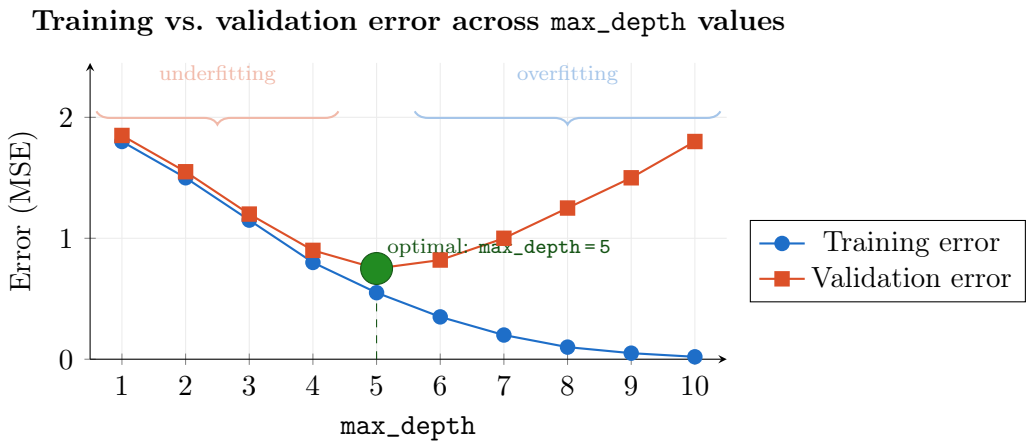


Figure 2: Validation curve for `max_depth` tuning. Training error decreases monotonically while the validation error follows a U-shape, reaching its minimum at `max_depth = 5`. The test set is not touched until after this selection.

The test MSEs obtained are shown below.

`max_depth`	**Train MSE**	**Test MSE**
1	1.82	1.79
2	1.41	1.38
3	0.97	0.94
4	0.61	**0.58**
5	0.28	0.74

She selects `max_depth = 4` (lowest test MSE = 0.58) and reports: "*Our model achieves a generalization error of 0.58 MSE.*"

Questions.

1. Why is the reported MSE of 0.58 optimistically biased?
2. In this process, how many times did the test set effectively influence a modeling decision?
3. How should the workflow be restructured to produce an unbiased estimate? Propose a corrected split and describe each step.

Solution.

Q1 — Why 0.58 is optimistically biased.

The practitioner used the test set *to choose* `max_depth = 4`. The winning configuration was selected precisely because it happened to score best on *this particular* set of 300 test examples. As a consequence:

- The 300 test examples were not neutral observers; they participated in the selection decision.

- The reported score reflects, in part, the fit between the model and those specific 300 points, not its expected performance on arbitrary unseen data.
- On average, evaluation on fresh unseen data would be expected to yield a worse score than 0.58.

In the table, note that `max_depth = 5` already shows a rising test MSE (0.74 vs. train MSE 0.28), a clear sign of overfitting. The test set "correctly" penalized depth 5, but only because it happened to be a representative sample at that boundary. Relying on a single split for both selection and reporting is risky.

Q2 — How many times did the test set influence a decision?

Five times: once per candidate model evaluated. Each evaluation of $J_{\text{test}}(\theta^{(d)})$ was a "vote" that contributed to the final selection. Even though the practitioner only made one explicit choice, the test set influenced five model comparisons.

As a general rule: if m candidate models are compared using the same test set, the effective number of "uses" of that test set is m, not 1. The more models compared, the larger the optimistic bias.

Q3 — Corrected workflow (three-way holdout).

1. **Split once:** partition the 1 000 examples into training (70%, 700 examples), validation (15%, 150 examples), and test (15%, 150 examples). Set the test set aside and do not touch it again until step 4.
2. **Tune on the validation set:** train a decision tree for each `max_depth` $\in \{1, 2, 3, 4, 5\}$ using the 700 training examples and compute J_{val} for each. Select the depth d^* that minimizes J_{val}. The test set plays no role here.
3. **Retrain:** fit the final model with d^* on the combined training + validation set (850 examples) to maximize the data available for the deployed model.
4. **Report:** evaluate the retrained model on the 150 test examples *exactly once*. This number is the unbiased generalization estimate.

	Wrong	**Correct**
Test set used to select model?	**Yes**	**No**
Effective number of test-set uses	5	1
Reported MSE	0.58 (biased low)	Unbiased
Validation set exists?	**No**	**Yes**

Key takeaway. Every model comparison that uses the test set for selection consumes one "use" of that set. The test set must be reserved for a single final evaluation, after all selection decisions have been made using a separate validation set.

4.4 Typical Split Proportions

There is no single correct way to divide a dataset into training, validation, and test subsets. The right proportions depend on how much data are available and on what each subset

needs to accomplish. Understanding the reasoning behind the common rules of thumb is more useful than memorizing the numbers themselves.

The guiding principle

Each of the three subsets has a distinct job, and each job requires a minimum amount of data to do it well.

- The **training set** needs to be large enough for the model to learn the underlying pattern. Too little training data and the model will underfit regardless of hyperparameter choices, because there simply is not enough signal to learn from.
- The **validation set** needs to be large enough to produce a reliable estimate of generalization error. An estimate based on very few examples has high variance: it will fluctuate substantially from one random split to another, making hyperparameter comparisons unreliable. Absolute counts matter more than proportions, but what is "enough" depends strongly on the task, class balance, and metric. For difficult or imbalanced problems, even a few hundred validation examples may be insufficient.
- The **test set** needs to be large enough to produce a statistically meaningful final performance report. If the test set is too small, the confidence interval around the reported metric will be wide, and two models with genuinely different performance may be indistinguishable.

These three requirements pull in different directions: every example given to validation or testing is one fewer example for training. The split proportions are the outcome of resolving this tension given the total number of available examples.

Small datasets

When the dataset contains at most a few thousand examples, each subset must be allocated a substantial fraction of the data or it will be too small to be useful. The splits **70/15/15** and **60/20/20** reflect this constraint: they reserve enough data for training while ensuring that both the validation and test sets contain enough examples for stable estimates.

The cost is real: a 70/15/15 split on a dataset of 1 000 examples leaves only 150 examples each for validation and testing. If the task is fine-grained (many classes, rare events, or high noise), even 150 examples may be insufficient. In those situations, k-fold cross-validation on the training and validation portions is the standard remedy, since it uses the available data more efficiently than a fixed holdout split.

Large datasets

When the dataset contains tens or hundreds of thousands of examples, the fractions needed for validation and testing shrink considerably. A 90/5/5 split on a dataset of 100 000 examples still yields 5 000 examples each for validation and testing, which is far more than enough for stable estimates. Keeping a larger fraction for training is generally beneficial here: the model sees more data, which generally improves generalization. There is no reason to "waste" data on a validation set larger than what is needed for reliable hyperparameter comparisons.

Dataset size	Typical split	Rationale
Small ($\lesssim$5k examples)	70/15/15 or 60/20/20	Reserves enough data for stable val. and test estimates despite limited total size.
Medium (5k–100k)	80/10/10	Balances training data against reliable holdout estimates.
Large ($\gtrsim$100k examples)	90/5/5 or 95/2.5/2.5	Small fractions already yield large absolute counts; maximizing training data pays off.

What the proportions do *not* tell you

Two important caveats are worth stating explicitly.

First, **proportions are a proxy for absolute counts**. A 15% validation set on a dataset of 200 examples is only 30 examples — likely insufficient. A 5% validation set on a dataset of 500 000 examples is 25 000 examples — more than enough. Always check the absolute sizes of each subset after splitting, not just the percentages.

Second, **the proportions assume a random, i.i.d. split**. When the data have temporal structure (e.g., time series), geographic clusters, or repeated measurements from the same subjects, a naive random split can leak information across subsets. In those cases, the split must respect the natural boundaries in the data — for instance, splitting by time (all data before date t for training, data after t for testing) or by subject (no subject appears in more than one subset). Correct split design often matters more than the exact proportions chosen.

Rule of thumb

Choose proportions so that the validation and test sets each contain at least a few hundred examples in absolute terms. If that is not possible while keeping a sufficiently large training set, switch to k-fold cross-validation rather than a fixed holdout split.

For classification problems, the splits should usually be **stratified**, meaning that each subset preserves approximately the same class proportions as the full dataset. This prevents validation or test sets from containing too few examples of rare classes.

4.5 Advantages and Limitations

Advantages: avoids optimistic bias; simple to implement and conceptually intuitive.

Limitations: the estimate still depends on a single random split; available data are reduced; for small datasets this reduction can noticeably increase variance.

Common Mistakes

- Reusing the test set multiple times during tuning.
- Forgetting to shuffle the data before splitting.
- Reporting validation error instead of test error as final performance.
- Applying preprocessing to the full dataset *before* splitting; this leaks test statistics into training.

Entry Ticket 3

A student splits a dataset 70/15/15, tunes hyperparameters using the validation set, and then reports the *validation* error as the model's final performance.

1. Is this estimate biased? In which direction?
2. What should the student report instead, and why?
3. When is it acceptable to retrain on the combined training + validation data before the final test evaluation?

5 k-Fold Cross-Validation

5.1 The Big Picture: Where Cross-Validation Fits

Before diving into the mechanics, it is essential to situate cross-validation within the broader data partitioning strategy used in modern machine learning practice. A common source of confusion is thinking that cross-validation *replaces* the test set. It does not.

The correct mental model is the following two-level partition:

1. The full dataset $\mathcal{D}$ is first split into two portions: a **development set** (also called the train+validation set) and a **final test set**. The test set is set aside immediately and treated as a sealed envelope; it will not be opened until all modeling decisions are complete.

2. Within the development set, cross-validation is used to tune hyperparameters, compare algorithms, and obtain a reliable estimate of generalization performance. The test set plays no role whatsoever in this process.

This distinction matters because it clarifies exactly what cross-validation is for: it is a smarter way to exploit the development data for model selection, not a way to avoid having a test set. Every workflow discussed from this point forward assumes this two-level structure.

Figure 3 summarizes this two-level structure. The full dataset $\mathcal{D}$ is first divided into a development set and a final test set; the test set is sealed immediately and plays no role during model development. Within the development set, k-fold CV replaces the fixed validation split of the three-way holdout: instead of permanently reserving a fraction for validation, the folds rotate across k iterations so that every example serves as a validation point exactly once. The averaged error E_{CV} is then used to guide model selection and hyperparameter tuning. Only after all such decisions are finalized is the test set opened (exactly once) to produce an unbiased estimate of the chosen model's generalization performance.

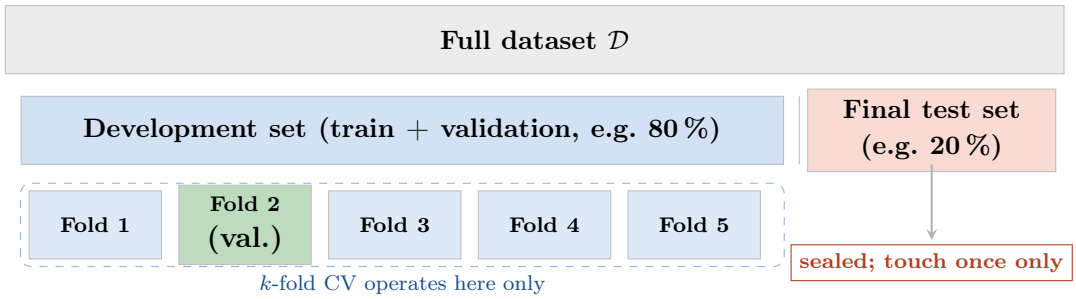


Figure 3: Two-level data partitioning in modern ML practice. The full dataset is first split into a **development set** and a **final test set**. The test set is sealed immediately and never consulted during model development. Within the development set, k-fold CV rotates the validation fold across k iterations, providing a stable estimate of generalization performance for model selection and hyperparameter tuning. Only after all decisions are finalized is the test set opened (exactly once) to produce an unbiased performance report.

5.2 Motivation

Within the development set, the three-way holdout dedicates a fixed fraction to validation (say, 20% of the development data) and the remainder to training. The estimate of generalization performance obtained this way depends critically on a *single* random partition: if the validation split happens to contain unusually easy or unusually hard examples, the measured error will be misleadingly low or high. For small or medium-sized datasets, this variability can be pronounced enough to mislead model selection.

The key idea of **k-fold cross-validation** is that, instead of committing to one fixed validation split within the development data, we average the performance estimate over k different, complementary partitions. Each partition uses a different slice of the development data as the validation set and the rest as the training set. The result is a substantially more stable and representative estimate.

An intuitive analogy can be made with estimating the expected value of a fair die. The true mean of a six-sided die is

$$\mathbb{E}[X] = 3.5.$$

Suppose we ask a student to roll the die once and use the observed value as an estimate of this mean. If the outcome is 6, the estimate becomes 6; if the outcome is 1, the estimate becomes 1. Clearly, such an estimate is extremely noisy.

Now suppose the student rolls the die 10 times and computes the average of the results:

$$\bar{X} = \frac{1}{10} \sum_{i=1}^{10} X_i.$$

This average will typically be much closer to 3.5. Importantly, even though both procedures produce unbiased estimates of the true mean, the second one has a much smaller variance:

$$\mathrm{Var}(\bar{X}) = \frac{\sigma^2}{n}.$$

The same principle applies to model evaluation. A single validation split produces one noisy estimate of the model's generalization error, just as a single die roll produces a noisy estimate of the mean. Methods such as k-fold cross-validation reduce this variance by averaging the performance over multiple train–validation partitions, leading to a more stable and reliable basis for model selection.

5.3 Concept and Procedure

The development set is divided into k approximately equal, disjoint subsets called **folds**: $\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_k$. The algorithm performs k training-and-evaluation rounds. In round i, fold $\mathcal{D}_i$ serves as the validation set while the remaining $k-1$ folds form the training set. Each fold is used exactly once as the validation set, so every example in the development set contributes to both training (in $k-1$ rounds) and validation (in exactly one round).

Procedure.

1. Randomly shuffle the development set and partition it into k disjoint folds $\mathcal{D}_1, \ldots, \mathcal{D}_k$ of roughly equal size.
2. For each $i \in \{1, \ldots, k\}$:
   (a) Train the model on $\mathcal{D} \setminus \mathcal{D}_i$ (all folds except $\mathcal{D}_i$).
   (b) Evaluate the trained model on $\mathcal{D}_i$ and record error E_i.
3. Compute the cross-validation estimate:

$$E_{\text{CV}} = \frac{1}{k} \sum_{i=1}^{k} E_i.$$

Because no single split dominates the estimate, E_{CV} is far less sensitive to random fluctuations in how the development data happen to be arranged.

Figure 4 illustrates the mechanics of 5-fold CV applied to the development set. The development data are partitioned into five folds of equal size; in each of the five iterations, one fold (green) serves as the validation set while the remaining four (blue) are combined to form the training set. The role of the validation fold rotates systematically: Fold 1 is held out in iteration 1, Fold 2 in iteration 2, and so on, until every example has been used for validation exactly once and for training in the other four iterations. Each iteration produces an error estimate E_i based on a different validation fold; the five estimates are then averaged to yield E_{CV}, which is substantially more stable than any individual E_i would be on its own.

5.4 Advantages and Limitations

5.4.1 Advantages

More reliable estimates. Because E_{CV} averages error estimates obtained from multiple validation folds, it is typically more stable than a single holdout estimate, although the fold estimates are correlated rather than independent. This is especially important for

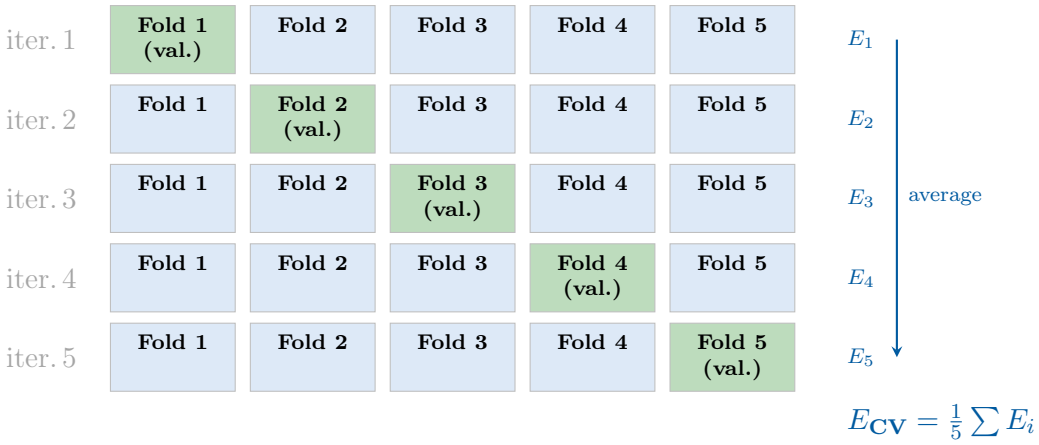


Figure 4: 5-fold cross-validation applied to the development set. In each of the five iterations, one fold (green) serves as the validation set while the remaining four (blue) form the training set. Every example participates in exactly one validation round. The five errors are averaged to yield the final estimate E_{CV}. The final test set (not shown) remains untouched throughout.

small datasets, where a single split can produce widely different results depending on the random seed.

More efficient use of the development data. In a three-way holdout, a fixed fraction of the development data is permanently assigned to validation and never used for training. In k-fold CV, every example in the development set is used for training in $k-1$ out of k rounds and for validation in exactly one round. The same data serve both purposes at different times, maximizing how much each example contributes to the modeling process.

Diagnostic power. Beyond the average E_{CV}, the individual fold errors $E_1, \ldots, E_k$ carry useful information. Their standard deviation is an empirical measure of how sensitive the model is to the particular training sample: a large spread suggests high variance and indicates that the model may benefit from more data or stronger regularization.

5.4.2 Limitations

Computational cost. The model must be trained k times instead of once. For expensive models (e.g., deep neural networks, large ensemble methods) this multiplied cost can be prohibitive.

Residual variance for small k. For very small k (e.g., $k = 2$), each training set contains only half the development data and the estimate can remain noisy.

Independence assumption. Standard k-fold CV assumes the examples are exchangeable (roughly, i.i.d.). When data have temporal, spatial, or group structure, naive random partitioning can produce overoptimistic estimates. Specialized variants (e.g., time-series CV, group k-fold) address these cases.

5.5 Choice of k

The choice of k governs a bias–variance trade-off *within* the cross-validation procedure itself, separate from the bias–variance trade-off of the model being evaluated.

- **Small k:** Each training set is a smaller fraction of the development data, so the models trained in each fold may underperform relative to a model trained on the full development set. This introduces pessimistic bias into the cross-validation estimate E_{CV}. The upside is lower computational cost.
- **Large k (up to $k = m$, leave-one-out):** Each training set is nearly as large as the full development set, reducing bias. However, computational cost grows linearly with k, and the k training sets become very similar to one another, which can paradoxically increase the variance of E_{CV} [Arlot and Celisse, 2010].

Dataset size	Common value	Rationale
Small (hundreds of samples)	$k = 5$ or $k = 10$	Reduces pessimistic bias; affordable cost
Medium to large	$k = 3$ or $k = 5$	Good balance; manageable overhead
Very large	$k = 2$ or holdout	Single split already stable enough

Empirical studies suggest $k = 10$ offers a good balance in most settings [Kohavi, 1995, Efron and Tibshirani, 1994]. The extreme case $k = m$, called **Leave-One-Out CV (LOOCV)**, is nearly unbiased but intractable for large datasets, requiring m separate training runs.

5.6 Relation to the Test Set

As established in Figure 3, k-fold CV operates entirely within the development set. The roles of each data partition are fixed from the moment the initial split is made:

Partition	Used in CV?	Purpose
Training folds	Yes (fitting)	Learn model parameters
Validation fold	Yes (scoring)	Estimate generalization during selection
Final test set	Never	Single unbiased performance report

The practical workflow is therefore:

1. Separate the final test set at the very beginning and put it aside.
2. Use k-fold CV on the remaining development data to select hyperparameters and compare models.
3. Retrain the chosen model on the entire development set.
4. Evaluate exactly once on the test set and report the result.

5.7 Cross-Validation for Hyperparameter Tuning

One of the most important applications of CV is systematic hyperparameter search. For each candidate configuration, perform k-fold CV on the development data and compute E_{CV}. After evaluating all configurations, select the one with the best average score. This process never involves the test set, so the test set retains its role as an unbiased judge.

Example: tuning an SVM. Suppose we tune $C \in \{0.01, 0.1, 1, 10, 100\}$ and $\gamma \in \{0.001, 0.01, 0.1\}$ for a Support Vector Machine using a grid search with 5-fold CV. This evaluates $5 \times 3 = 15$ pairs, each on 5 folds, requiring 75 training runs in total. The pair with the lowest E_{CV} is selected.

5.8 Cross-Validation for Algorithm Selection

The same mechanism extends to comparing different learning algorithms. Each algorithm (with its best hyperparameter settings already determined) is evaluated through CV on the same development folds. The algorithm with the best average CV score is selected as the candidate for final evaluation on the test set.

When comparing algorithms, it is important to use the *same* folds for all candidates. This ensures that differences in E_{CV} reflect genuine differences in algorithm quality rather than differences in which examples happened to land in the validation fold. In scikit-learn, this is achieved by constructing a single `KFold` object with a fixed random state and reusing it across all evaluations.

5.9 When Cross-Validation Is Not Enough

Using k-fold CV both for hyperparameter tuning and for reporting the final performance estimate can still produce overoptimistic results. The reason is subtle but important: when we search over many configurations and select the one with the lowest E_{CV}, we are fitting a meta-level model to the validation folds. The winning configuration was chosen precisely because it performed well on those folds; its CV score is therefore a biased estimator of true generalization [Cawley and Talbot, 2010]. The more configurations we compare, the more pronounced this bias becomes.

This is the same fundamental problem as using a test set for selection: whichever option "wins" does so partly due to random good luck on the held-out data, and reporting that score as the final estimate overstates real-world performance.

The solution is to maintain two separate CV loops: an **inner loop** dedicated to hyperparameter tuning, and an **outer loop** dedicated to unbiased performance estimation. This strategy, **nested cross-validation**, is the topic of the next section.

6 Nested Cross-Validation

6.1 Motivation

Using k-fold CV for both hyperparameter tuning and model evaluation can yield an overly optimistic performance estimate, because the same data influence both activities. **Nested cross-validation (nested CV)** solves this by separating the two tasks:

- an **outer loop** that estimates generalization performance;
- an **inner loop** that performs hyperparameter tuning.

Because the outer test folds are never seen by the inner loop, the outer estimate remains approximately unbiased even in the presence of systematic hyperparameter search.

6.2 Outer Loop: Evaluation

1. Split the dataset into k_{out} folds.
2. For each fold i:
   - Hold out fold i as the outer test set.
   - Use the remaining $k_{\text{out}} - 1$ folds as the outer training set.
   - Run the inner loop on the outer training set.
   - Evaluate the resulting model on fold i; record score S_i.
3. Compute the final estimate: $\bar{S} = \frac{1}{k_{\text{out}}} \sum_{i=1}^{k_{\text{out}}} S_i$.

6.3 Inner Loop: Hyperparameter Tuning

1. On the current outer training set, perform k_{in}-fold CV.
2. Evaluate all candidate hyperparameter configurations.
3. Select the configuration λ^* with the best inner validation score.
4. Retrain the model on the full outer training set using λ^*.

Figure 5 illustrates the procedure of nested cross-validation, separating clearly the roles of model evaluation and hyperparameter selection. In each outer iteration, the dataset is partitioned into (k_{out}) folds. One fold (shown in red) is held out as the outer test fold, while the remaining folds (shown in blue) form the outer training data. The model is never trained on the outer test fold during that iteration; instead, it is used exclusively to compute a performance score (S_i). The dashed rectangle surrounding the blue folds in the first row highlights the portion of the data that is available for training in that outer iteration. From this subset, a second procedure, called the *inner cross-validation loop*, is constructed. The lower dashed box represents this inner loop, where the outer training data are further partitioned into (k_{in}) folds that alternate between training and validation roles. Across these inner iterations, candidate hyperparameters (e.g., (λ)) are evaluated, and the configuration that yields the best validation performance is selected. Once the optimal hyperparameters (λ^*) are determined, the model is retrained using the entire outer training set (all blue folds in the upper box) and then evaluated on the outer test fold, producing the score (S_1). The process is repeated for each outer fold (e.g., in the second row, Fold 2 becomes the outer test set), yielding scores $(S_1, S_2, \ldots, S_{k_{\text{out}}})$. Averaging these scores provides an estimate of the model's generalization performance that properly accounts for hyperparameter tuning, avoiding the optimistic bias that would arise if the same data were used simultaneously for both model selection and final evaluation.

Entry Ticket 4

Nested CV with $k_{\text{out}} = 5$ and $k_{\text{in}} = 3$ is run to compare two algorithms, each with a grid of 10 hyperparameter values.

1. How many total model trainings are required?
2. The inner loops selected `max_depth = 6` in 4 of the 5 outer iterations and `max_depth = 5` once. Which value would you use for the final deployed model? Why?

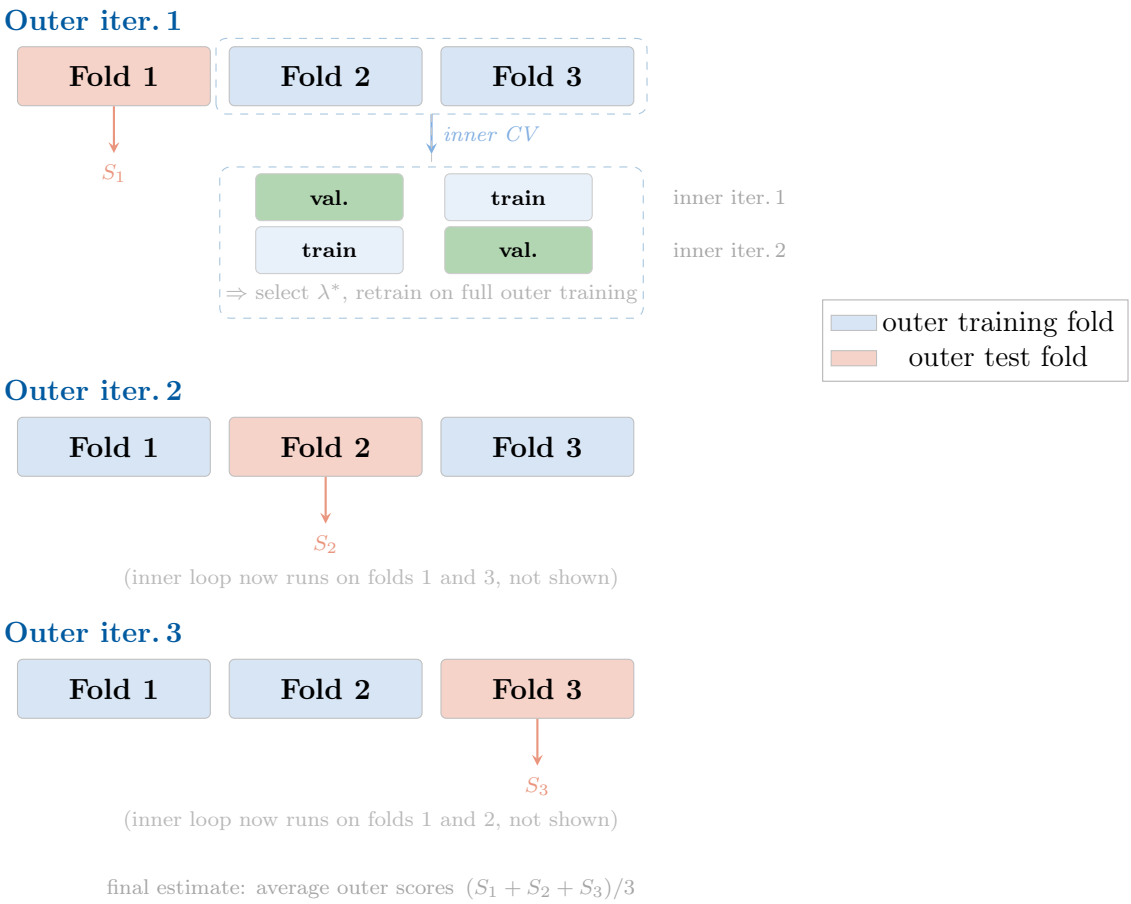


Figure 5: Nested cross-validation ($k_{\text{out}} = 3$, $k_{\text{in}} = 2$). In each outer iteration, one fold (red) is held out as the outer test set, while the remaining folds (blue) form the outer training data used for model selection inside the inner cross-validation loop. The first outer iteration is shown in detail: the blue folds are isolated, passed to the inner CV procedure, used to select the best hyperparameter setting λ^*, and then used again to retrain the model before evaluation on the held-out outer test fold. The same logic is repeated across all three outer iterations, yielding scores S_1, S_2, and S_3, whose average provides a less biased estimate of generalization performance.

3. What is the key property that makes the outer test scores unbiased?

6.4 Illustrative Example

To make the procedure concrete, consider the situation depicted in Figure 5, where $k_{\text{out}} = 3$ and $k_{\text{in}} = 2$.

- In the **first outer iteration**, Fold 1 is held out as the outer test set, while Folds 2 and 3 form the outer training set.
- The outer training set is then passed to the inner cross-validation loop. With $k_{\text{in}} = 2$, the data are split into two folds that alternate between training and validation roles.
- Suppose we are tuning a hyperparameter λ with candidate values $\{0.01, 0.1, 1\}$. Each value is evaluated using the inner cross-validation procedure.

- The configuration with the best inner validation score is selected (e.g., $\lambda^* = 0.1$).
- The model is then retrained on the full outer training set (Folds 2 and 3) using λ^* and evaluated on the outer test fold (Fold 1), producing the score S_1.

The same procedure is repeated in the remaining outer iterations: Fold 2 becomes the outer test set in the second iteration (yielding S_2), and Fold 3 becomes the outer test set in the third iteration (yielding S_3). The final performance estimate is obtained by averaging the outer scores:

$$\bar{S} = \frac{S_1 + S_2 + S_3}{3}.$$

Because the outer test folds are never used during hyperparameter selection, this estimate provides a substantially less biased estimate of generalization performance.

6.5 Worked Example: Tracing Nested CV Step by Step

Consider the following scenario. We apply nested CV with $k_{\text{out}} = 3$ outer folds and $k_{\text{in}} = 2$ inner folds to tune a decision tree regressor. The only hyperparameter is `max_depth` $\in \{2, 4, 6\}$. The dataset has 180 examples, split into three outer folds of 60 examples each. The table below records the *inner* fold validation MSEs (already averaged over the 2 inner folds) and the *outer* test MSE for each combination.

Inner CV results (mean MSE over 2 inner folds).

Outer iter.	`depth=2`	`depth=4`	`depth=6`
1 (test = Fold 1)	1.42	**0.91**	1.05
2 (test = Fold 2)	1.39	**0.88**	1.10
3 (test = Fold 3)	1.45	0.95	**0.92**

Outer test MSE after retraining with λ^*.

Outer iter.	Selected λ^*	Train set size	Outer test MSE (S_i)
1	`depth=4`	120	0.97
2	`depth=4`	120	0.93
3	`depth=6`	120	1.01

Questions.

1. How many total model trainings did this nested CV require?
2. What is the final nested-CV performance estimate $\bar{S}$?
3. `max_depth=4` was selected in 2 of 3 outer iterations. Which value should be used for the final deployed model? Why?

4. Why is $\bar{S}$ an unbiased estimate of generalization error, whereas the E_{CV} from a single (non-nested) CV loop would not be?

Solution.

(Q1) Total model trainings.

Each outer iteration runs the inner CV loop plus one final retraining.

Step	**Per outer iter.**	**Total** ($\times 3$)
Inner CV: 3 configs $\times$ 2 folds	6	18
Retrain with λ^* on outer train set	1	3
Grand total		**21**

(Q2) Final performance estimate.

$$\bar{S} = \frac{S_1 + S_2 + S_3}{3} = \frac{0.97 + 0.93 + 1.01}{3} = \frac{2.91}{3} = \mathbf{0.97} \text{ MSE}.$$

This is the number that should be reported as the model's expected generalization error. It is obtained by averaging three outer-fold scores, each computed on data that were never seen by the inner tuning loop.

(Q3) Which λ^* for the deployed model?

`max_depth=4` was selected by the inner loop in 2 out of 3 outer iterations; `max_depth=6` was selected once (in outer iteration 3, where it narrowly outperformed depth 4: 0.92 vs. 0.95).

The recommended choice is `max_depth=4`, for two reasons:

- **Majority vote**: it was the winner in 2/3 iterations, suggesting it is the more reliably good configuration across different data samples.
- **Parsimony**: when two configurations perform similarly, the simpler one (shallower tree) is preferred. It has lower variance and is less likely to overfit on the full training set.

The final model is then retrained from scratch on all 180 examples using `max_depth=4`. The reported generalization error is still $\bar{S} = 0.97$, not the error of this final retrained model.

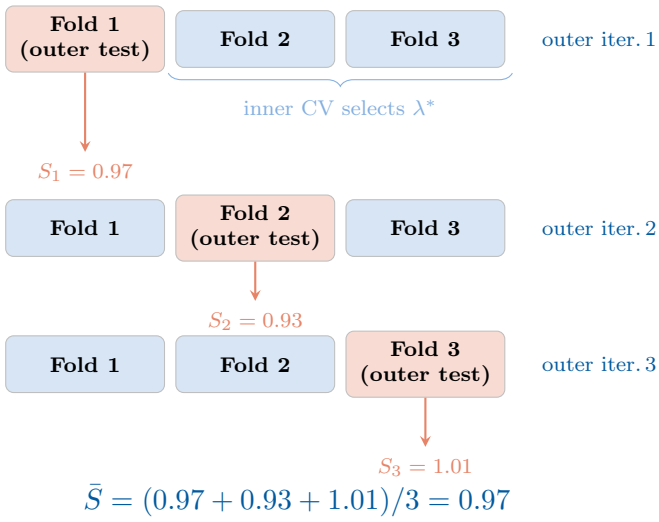
(Q4) Why $\bar{S}$ is unbiased.

In a single (non-nested) CV loop, the same folds are used both to select the best `max_depth` and to compute E_{CV}. The winning configuration was chosen *because* it scored best on those folds; its score is therefore an underestimate of its true error (optimistic bias).

In nested CV, the outer test folds are *shielded* from the inner selection loop:

- The inner loop sees only the outer training data (120 examples) and uses them to choose λ^*.

- The outer test fold (60 examples) is then used to evaluate the model retrained with λ^*, but it played *no role* in selecting λ^*.
- Because no selection decision was made using the outer test fold, its score S_i is an unbiased estimate of the generalization error of the procedure "run inner CV, pick best config, retrain."



Key takeaway. In nested CV, the outer test folds are never touched by the inner tuning loop. This two-level shielding is the only mechanism that simultaneously (a) searches for good hyperparameters and (b) provides an unbiased estimate of how well that search procedure generalizes. The reported number $\bar{S}$ describes the *procedure*, not just one particular model.

6.6 What About the Final Model?

Nested CV provides an estimate of generalization performance; it does not directly produce a single deployable model. Once nested CV is complete, a final model can be built in one of three ways:

1. Retrain on the full dataset using the hyperparameter configuration that performed best on average across the inner loops (or the configuration most frequently selected). This is the most common choice.
2. Perform one last inner search on all data to confirm the best configuration, then retrain from scratch.
3. Ensemble the outer-fold models by averaging or voting predictions (less common but can reduce variance).

In all cases, the reported generalization error should come from the nested CV process, **not** from the final retrained model, since only nested CV provides an unbiased estimate.

Computational cost

With k_{out} outer folds, k_{in} inner folds, and H hyperparameter configurations, nested CV requires approximately

$$k_{\text{out}}(k_{\text{in}} \times H + 1)$$

model trainings: $k_{\text{in}} \times H$ in the inner loop plus one retraining step per outer iteration.

Figure 6 summarizes the relationship between nested cross-validation and the construction of a final deployable model. The process begins with the full dataset, which is used to run the nested cross-validation procedure. The nested CV step produces an unbiased estimate of generalization performance, obtained by averaging the scores from the outer folds. Importantly, this estimate is conceptually separate from the training of the final model. Once evaluation is complete, the practitioner must still build a deployable model. This is typically done by retraining the model on the entire dataset using the hyperparameter configuration identified during the inner loops. Alternatively, the models trained in the outer folds may be combined into an ensemble. The key idea highlighted in the figure is that nested cross-validation serves primarily as an evaluation mechanism, while the final model is constructed only after that evaluation step has been completed.

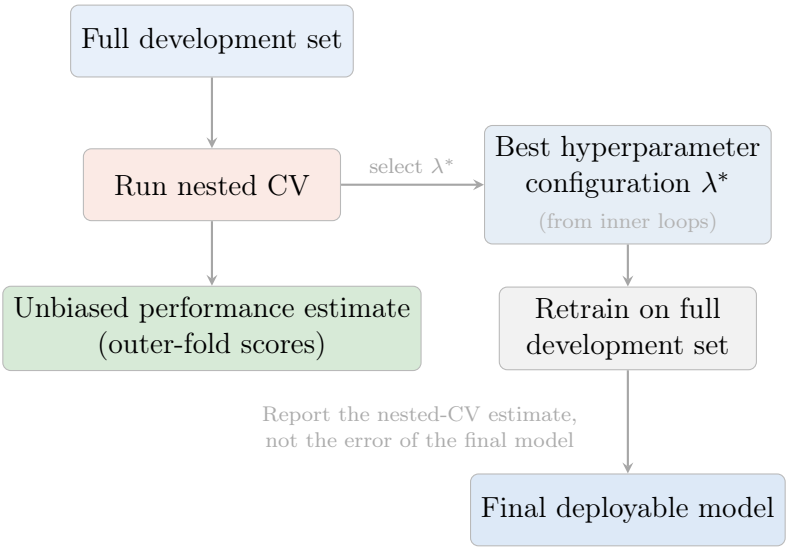


Figure 6: Relationship between nested cross-validation and the final deployable model. Nested CV serves as an *evaluation* mechanism: it produces an unbiased estimate of generalization performance from the outer-fold scores. Separately, the inner loops identify the best hyperparameter configuration λ^*, which is then used to retrain the model on the full development set. The reported performance figure must come from the nested-CV estimate, not from evaluating the final retrained model on the data it was trained on.

7 Final Remarks

7.1 Beyond the Methods Presented

Throughout these notes we covered: testing on training data (not recommended), two-way holdout, three-way holdout, k-fold CV, and nested CV. Other useful approaches include:

- **Leave-one-out CV (LOOCV):** uses a single observation as the test set in each iteration. With n observations, the procedure performs n training runs, each time

leaving out one point for testing. LOOCV makes very efficient use of the available data but can be computationally expensive and may produce high-variance performance estimates.

- **Stratified k-fold CV:** preserves class proportions in each fold, especially important for imbalanced classification problems. Without stratification, some folds might contain very few or even no examples of a minority class, leading to unstable or misleading performance estimates.
- **Bootstrap validation:** draws multiple resamples (with replacement) from the dataset to create pseudo-training sets. Observations not included in a particular resample (the "out-of-bag" samples) serve as test data. This approach is useful when the dataset is very small and traditional splitting would leave too little data for training [Efron and Tibshirani, 1994].
- **Time-series cross-validation:** designed for temporally ordered data, where randomly shuffling observations would break the causal structure of the problem. Instead, training sets are formed using earlier observations and evaluated on later ones. A common strategy is the *expanding window*, where the training set grows over time (e.g., train on $t_1 : t_k$, test on t_{k+1}), or the *rolling window*, where both training and testing move forward through time.
- **Group k-fold CV:** used when observations are not independent but belong to identifiable groups (e.g., patients, users, documents, or geographic locations). In this scheme, all samples from the same group are kept within the same fold, ensuring that the model is not evaluated on data that are strongly related to the training examples. This prevents overly optimistic estimates caused by information leakage across groups.

General principle. All these methods attempt to estimate how well a model trained on a sample of data will generalize to unseen examples.

Practical guidance. The choice of validation strategy should reflect the structure of the dataset and the assumptions of the learning problem.

- **Standard k-fold CV** works well when observations are approximately independent and identically distributed (i.i.d.).
- **Stratified k-fold CV** is usually preferred for classification tasks, especially when class imbalance is present.
- **Group k-fold CV** should be used when multiple observations belong to the same entity (e.g., measurements from the same patient, transactions from the same user, or images from the same subject). This prevents leakage between training and validation data.
- **Time-series cross-validation** is appropriate whenever the data have a temporal ordering (e.g., forecasting, sensor streams, financial data). Training must always use past observations to predict future ones.
- **LOOCV or bootstrap methods** may be useful when the dataset is extremely small and it is desirable to train the model on nearly all available observations in

each iteration.

In practice, the goal is always the same: simulate as faithfully as possible the conditions under which the model will be used after deployment. Whenever the validation procedure violates the structure of the data (e.g., mixing future observations into the training set or splitting related samples across folds), the estimated performance may become optimistically biased.

7.2 Which Method Should You Use?

There is no universal recipe, but the following guidelines apply in most situations:

- Use two-way or three-way holdout for quick experiments and large datasets.
- Use k-fold CV when data are limited and a stable performance estimate is needed.
- Use nested CV when hyperparameter tuning and algorithm selection are involved, or when reporting final model performance in academic work.

Trade-off. As we move from simpler to more sophisticated methods:

$$\text{train-only} \rightarrow \text{two-way holdout} \rightarrow \text{three-way holdout} \rightarrow k\text{-fold CV} \rightarrow \text{nested CV}$$

(increasing reliability and computational cost)

7.3 Do You Always Need a Separate Test Set?

Throughout these notes, the two-level partitioning strategy (development set plus a sealed final test set) was presented as the standard workflow. This is the right default for holdout-based methods and for plain k-fold CV, where the validation mechanism does not itself provide an unbiased estimate of generalization error. In those cases, the test set is essential: without it, there is no data partition whose role is exclusively evaluation, and any performance figure reported will be optimistically biased to some degree.

Nested CV changes this situation. The outer loop of nested CV is designed precisely to produce an unbiased estimate of generalization performance: the outer test folds are never seen by the inner tuning loop, so the averaged outer-fold score $\bar{S}$ has the same statistical role as the error measured on a held-out test set. When nested CV is applied to the full dataset, a separate test set is therefore **conceptually redundant**; the outer loop already provides what the test set would otherwise provide.

In practice, the choice depends on the goal and the available data:

- **Nested CV without a separate test set.** When the dataset is small, or when the goal is to report the expected performance of the modeling *procedure* (as is common in academic work comparing algorithms) applying nested CV to the full dataset is the preferred approach. It wastes no data on a permanently held-out fraction and produces the most reliable estimate available.
- **Nested CV with a separate test set.** When the dataset is large, when regulatory or institutional protocols require a fully independent evaluation, or when there is concern about subtle data leakage that is difficult to rule out, maintaining a separate test set provides an additional layer of assurance. In this case, nested CV operates

on the development set and the test set is used only for the final report, exactly as described earlier in these notes.

The key principle underlying both choices is the same: *the data used to select a model cannot also be used to evaluate it without introducing bias.* Nested CV enforces this separation by construction, through its two loops. A held-out test set enforces it by physical partition. Either mechanism suffices; the choice between them is a matter of data availability and practical context, not of conceptual correctness.

7.4 Choosing the Right Metric

Selecting the appropriate evaluation metric is as important as choosing the right validation method:

- In classification: accuracy, precision, recall, F1 score, or AUC depending on the application.
- In regression: MSE, RMSE, MAE, and R^2.

Practical rule. Always align the metric with the goal of the application. In medical diagnosis, recall (sensitivity) is often more important than precision, because missing a positive case can have severe consequences. In spam detection or fraud alerting, precision tends to be more important, since false positives (marking a legitimate email as spam, or flagging a valid transaction as fraud) can degrade user trust.

7.5 Keep the Test Set in a Safe

One of the most common and consequential mistakes in applied machine learning is consulting the test set more than once. Every time the test set is used to guide a decision (even informally, such as noticing that one model "looks better" on it and deciding to keep tuning), it transitions from being a neutral judge to being an active participant in the training process. From that moment on, the reported test score is no longer an unbiased estimate of generalization: it reflects, at least in part, choices that were made *because* of how the model performed on those exact examples.

This contamination is subtle and cumulative. Suppose a practitioner trains ten models, checks the test error after each one, and picks the best. The winner was selected precisely because it happened to perform well on that particular sample of test data, which may not be representative of the broader population. Using the observed test score to influence decisions increases the probability that the reported score is a statistical artefact rather than a true reflection of how the model will behave in deployment.

François Chollet captures this dynamic concisely:

> *"Most people think of a validation set as a weak test set, but it is more accurately a weak training set. Data that you use to improve your model are data on which your model will perform artificially well."*
>
> — *François Chollet* [Chollet, 2021]

The principle generalizes: **any data used to make a decision about the model becomes, in effect, training data.** The test set loses its value the moment it influences

a choice, whether that choice is a hyperparameter value, an architectural decision, a preprocessing step, or even the decision to stop experimenting.

For CV and nested CV the more important methodological lesson is even more specific: *all preprocessing steps that learn from data must be fitted inside each training fold.* In cross-validation settings, the entire pipeline must be refit inside each fold. Any preprocessing step that uses information from the data must be learned on the training portion of the fold only, then applied to the validation portion.

Pipeline rule for cross-validation

Any preprocessing step that learns from the data (e.g., scaling, imputation, PCA, feature selection, target encoding) must be fitted using only the training portion of each fold and then applied to the validation portion. Fitting preprocessing on the full dataset before cross-validation leaks information and produces overly optimistic estimates.

The practical rule. Treat the test set as a sealed envelope. Open it exactly once, only after all training, validation, and hyperparameter tuning are fully complete, and report whatever number comes out, whether it is encouraging or not. If the result is disappointing and prompts further adjustments, the test set must be retired and replaced with fresh data before any new evaluation is meaningful.

This discipline applies regardless of the validation strategy used. In a three-way holdout, the test set is sealed from the start. In k-fold CV, a held-out test portion remains sealed while the folds serve as the validation mechanism. In nested CV, the outer test folds are shielded from the inner tuning loop by construction. In all cases, the test data must remain completely untouched until the final step.

7.6 Final Takeaways

Key Takeaways

- Model selection is a crucial step that connects model evaluation with real-world deployment.
- The goal is to find a model that generalizes well, not merely one that fits the training data perfectly.
- Keep clear boundaries between training, validation, and test data.
- Use cross-validation or nested cross-validation when high reliability is required.
- In model selection workflows with a held-out test set, evaluate once on that test set after retraining.
- In workflows based entirely on nested CV, report the outer-loop estimate, then retrain the deployable model on all available data.

7 References

Sylvain Arlot and Alain Celisse. A survey of cross-validation procedures for model selection. *Statistics Surveys*, 4:40–79, 2010.

Gavin C. Cawley and Nicola L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11:2079–2107, 2010.

François Chollet. *Deep Learning with Python*. Manning Publications, Shelter Island, NY, 2 edition, 2021.

Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, New York, 1994.

Ron Kohavi. A study of cross-validation and bootstrap for accuracy estimation and model selection. pages 1137–1143, 1995.